OLIVE: Lightweight Latent Residual Reinforcement Learning for Vision-Language-Action Models via Action Basis Projection

Gibeom Kim[1] and Sangmoon Lee[1*]

[1] School of Electronics and Electrical Engineering, Kyungpook National University, Daegu, 41566, Korea (gibeom6358@knu.ac.kr, sangmoon@knu.ac.kr)

* Corresponding author

Abstract: Vision-Language-Action (VLA) models possess powerful pre-trained representations for robotic manipulation; however, online reinforcement learning (RL) adaptation to new environments is computationally prohibitive, as it requires updating billions of parameters. In this paper, we propose **OLIVE (Online Latent In-situ VLA Enhancement)**, a framework that performs PPO-based adaptation by injecting latent residuals into the hidden state of a 7B-scale OpenVLA backbone, while keeping the backbone entirely frozen. Specifically, a 256-dim latent code z is projected through a basis matrix constructed from the action token rows of the frozen LM head, generating a 4096-dim Δh that is added to the backbone hidden state. This design constrains all perturbations to lie within the semantic action subspace already encoded by the language model. OLIVE trains only 300K parameters (0.004% of 7.3B total), and achieves 91.9% success rate (SR) on LIBERO-Spatial (+31.9pp over baseline), 71.4% on LIBERO-Goal (+1.4pp), and 50.0% on LIBERO-10 (+10.0pp). This work presents a novel direction for lightweight online VLA adaptation by repurposing the pre-trained structural knowledge of frozen LM heads for RL head design.

Keywords: Artificial Intelligence Systems; Robot Mechanism and Control; Robotic Applications

1. INTRODUCTION

Large-scale pre-trained Vision-Language-Action (VLA) models have demonstrated remarkable performance in following natural language instructions for complex robotic manipulation tasks. These models leverage powerful visual-linguistic representations learned from hundreds of millions of demonstrations, and supervised fine-tuning (SFT) has become the dominant paradigm for task-specific adaptation.

However, SFT-based adaptation has fundamental limitations. Real-world robotic environments exhibit distribution shifts in lighting conditions, object placement, and physical properties that differ from data collection settings, making SFT models brittle to such variations. While reinforcement learning (RL) has emerged as a promising approach for online adaptation, existing methods either update billions of parameters or require separate large critic networks, resulting in prohibitive computational costs.

In this paper, we pose the following central question: "Can the action representation space already encoded in a frozen VLA backbone be repurposed for RL adaptation?"

We answer this affirmatively by proposing OLIVE. The key insight is that the frozen LM head of a VLA model already encodes high-dimensional representations of action tokens — what we term the action basis. Unlike existing methods that add residuals in the low-dimensional action output space (7-dim) or update the entire model, OLIVE injects residuals at the hidden state level (4096-dim) while constraining the direction of perturbation to lie within the frozen LM's semantic action subspace. This allows RL to modify the model's "reasoning" rather than merely its "output," while preserving the general representational capacity of the backbone.

The main contributions of this paper are as follows:

Action Basis Projection: A novel architecture that repurposes the action token rows of the frozen LM head as a basis matrix, constraining latent perturbations to the semantic action subspace of the pre-trained model.

Extreme Parameter Efficiency: Only 300K parameters (0.004% of 7.3B) are trained, reducing computational cost by orders of magnitude compared to existing VLA+RL methods.

Multi-Suite Empirical Analysis: Systematic experiments across four LIBERO suites characterizing both the strengths and limitations of the proposed approach.

2. RELATED WORK

2.1 Vision-Language-Action Models

VLA models that combine large language models (LLMs) with visual encoders have established a new paradigm for instruction-following robotic manipulation. OpenVLA [1] integrates a LLaMA-7B backbone with a CLIP visual encoder, demonstrating strong zero-shot generalization across diverse manipulation tasks. OpenVLA-OFT [2] further improves inference speed and manipulation precision through parallel decoding and continuous action representations. However, both rely exclusively on SFT-based adaptation and lack mechanisms for online adaptation to environmental changes.

2.2 Reinforcement Learning for VLA Adaptation

Recent work has explored applying RL to VLA models to enable online adaptation. VLA-RL [3] proposes end-to-end PPO fine-tuning of the entire VLA model, but suffers from high memory requirements and training instability due to full-model updates. SimpleVLA-RL [4] improves efficiency by restricting RL training to the action head alone, though this limits representational capacity by leaving the backbone's hidden representations unchanged. TT-VLA [5] approaches adaptation from a test-time perspective, performing gradient updates at inference time, but requires a backward pass at every step, making it unsuitable for real-time deployment. In contrast, OLIVE keeps the backbone entirely frozen and achieves both expressiveness and efficiency through hidden-state-level residual injection.

2.3 Residual Reinforcement Learning

Residual RL [6] learns a residual policy on top of a pre-trained prior, providing an effective framework for adapting to new tasks while preserving prior knowledge. Whereas existing Residual RL methods define residuals in the low-dimensional action space, OLIVE injects residuals in the high-dimensional hidden state space (4096-dim), enabling adaptation within the language model's semantic representation space — a fundamentally different operating regime.

2.4 Parameter-Efficient Fine-Tuning

PEFT methods such as LoRA [7] enable efficient adaptation of large models by training only a small fraction of parameters. While LoRA-based SFT fine-tuning has been applied to VLA models, its combination with RL remains non-trivial due to training instability. OLIVE achieves extreme parameter efficiency (0.004%) in the RL setting while maintaining training stability through action basis projection, and can be viewed as a PEFT-inspired approach extended to the RL domain.

3. METHOD

3.1 Problem Formulation

We formulate robotic manipulation as a Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$. The state s_t consists of an RGB image observation $o_t \in \mathbb{R}^{256\times256\times3}$ and a natural language task instruction l. The action $a_t \in \mathbb{R}^7$ encodes end-effector position, orientation, and gripper state. The objective is to learn a policy π maximizing cumulative reward $\mathbb{E}[\sum_{t=0}^{T} \gamma^t r_t]$.

The central constraint in our setting is that the 7B-scale VLA backbone f_θ remains entirely frozen throughout training. Only the lightweight PPO head ϕ is optimized:

$$\pi_\phi(a_t|s_t) = \text{OLIVE}_\phi\big(f_\theta(o_t, l)\big) \tag{1}$$

3.2 VLA as a Frozen Feature Extractor

Given an observation-instruction pair, the frozen backbone processes the input as follows:

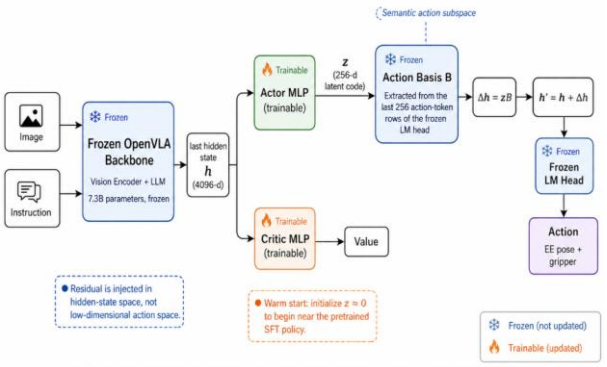


Fig 1. overall architecture of OLIVE

$$v = \text{VisionEncoder}(o_t) \in \mathbb{R}^{d_v} \tag{2}$$
$$h = \text{LLaMA}(v, l) \in \mathbb{R}^{4096} \tag{3}$$

The hidden state h encodes the distilled visual-linguistic representation of the backbone and serves as the sole input to the OLIVE head.

Action Basis Extraction. The LM head consists of a vocabulary projection matrix $W \in \mathbb{R}^{V\times4096}$. We extract the last 256 rows corresponding to action tokens to construct the action basis matrix:

$$B = W[-256:, :] \in \mathbb{R}^{256\times4096} \tag{4}$$

Each row of B defines a 4096-dim direction vector associated with a specific action token, collectively spanning the **semantic action subspace** that the backbone uses to represent actions. This basis is fixed throughout training.

3.3 Latent Residual Injection

Rather than adding residuals in the low-dimensional action output space as in conventional Residual RL, OLIVE injects residuals directly into the hidden state space. The key design principle is to constrain the direction of perturbation to lie within the row space of the action basis B, ensuring that RL operates exclusively within the backbone's semantic action representation. The Actor MLP maps the hidden state $h$to a 256-dim latent code z:

$$z = W_2 \cdot \text{ReLU}(W_1 h) \in \mathbb{R}^{256} \tag{5}$$

where $W_1 \in \mathbb{R}^{256\times4096}$ and $W_2 \in \mathbb{R}^{256\times256}$. The latent code is then projected through the action basis to produce the hidden state delta:

$$\Delta h = z \cdot B \in \mathbb{R}^{4096} \tag{6}$$

The modified hidden state is computed as:

$$\mathrm{h}' = \mathrm{h} + \alpha \cdot \Delta \mathrm{h} \tag{7}$$

where $\boldsymbol{\alpha}$ is the residual scale (set to 1.0 in our experiments). The final action is generated by passing

$\boldsymbol{h}'$through the frozen LM head:

$$a' = \text{LM_Head}(h') \in \mathbb{R}^7 \tag{8}$$

The output layer of the Actor is initialized with a gain of 0.01, ensuring $\boldsymbol{z} \approx \mathbf{0}$ and thus $\boldsymbol{\Delta h} \approx \mathbf{0}$ at the start of training. This warm start allows the policy to begin from the pre-trained SFT behavior and gradually deviate as learning progresses.

3.4 PPO Training

The Critic MLP independently estimates the state value from the same hidden state $\boldsymbol{h}$:

$$V(s) = W_4 \cdot \text{ReLU}(W_3 h) \in \mathbb{R}^1 \tag{9}$$

where $\boldsymbol{W_3} \in \mathbb{R}^{256\times4096}$ and $\boldsymbol{W_4} \in \mathbb{R}^{1\times256}$. The Actor and Critic share the input $\boldsymbol{h}$ but maintain independent parameters.

Advantage estimates are computed via Generalized Advantage Estimation (GAE), and the policy is updated using the clipped surrogate objective:

$$\mathcal{L}^{CLIP}(\phi) = \mathbb{E}_t[min(r_t(\phi)\hat{A}_t, \; clip(r_t(\phi), 1-\epsilon, 1+\epsilon)\hat{A}_t)] \tag{10}$$

The full training objective is:

$$\mathcal{L}(\phi) = -\mathcal{L}^{CLIP} + c_v\mathcal{L}^{VF} - c_e\mathcal{H}[\pi_\phi] \tag{11}$$

where $c_v = 0.5$ and $c_e = 0.01$.

The training procedure is summarized in Algorithms 1 and 2.

Input : $o_t, l,$ frozen f_θ, B
$h \leftarrow \underbrace{\text{LLaMA}(\text{CLIP}(o_t), l)}_{\text{no gradient}}$
$z \leftarrow W_2 \cdot \text{ReLU}(W_1 h)$
$\Delta h \leftarrow \underbrace{z \cdot B}_{\text{project onto action subspace}}$
$h' \leftarrow h + \alpha \cdot \Delta h$
$a' \leftarrow \underbrace{\text{LM-Head}(h')}_{\text{frozen}}$
$V(s) \leftarrow W_4 \cdot \text{ReLU}(W_3 h)$
return $a', V(s)$

Algorithm 1. OLIVE forward pass

3.5 Reward Design

The reward function is designed for compatibility with real-world deployment:

$$r_t = \begin{cases} r_{success} & if\ task\ completed \\ -\lambda \cdot t & otherwise \end{cases} \tag{12}$$

Task completion is determined using the simulation done signal in the LIBERO environment. The step penalty discourages timeout behavior. For real-world deployment, the task completion signal can be replaced by a proprioception-based proximity reward without modifying the RL training pipeline.

Initialize ϕ via warm start from SFT checkpoint
$B \leftarrow \underbrace{W_{\text{LM-Head}}[-256:,:]}_{\text{fixed throughout training}}$
for step = 1 to 5000 *do*
  task ← RoundRobin($\mathcal{E}$, rollouts_per_task = 5)
  for i = 1 to $|\mathcal{B}|$ *do*
    $a'_t, V(s_t) \leftarrow \text{OLIVEForward}(o_t, l)$
    $s_{t+1}, r_t, \text{done}_t \leftarrow \mathcal{E}.\text{step}(a'_t)$
  end for
  $\hat{A}_t \leftarrow \text{GAE}(\gamma = 0.99, \lambda = 0.95)$
  for epoch = 1 to 4 *do*
    $\mathcal{L} \leftarrow -\mathcal{L}^{\text{CLIP}} + c_v\mathcal{L}^{\text{VF}} - c_e\mathcal{H}$
    $\phi \leftarrow \phi - \eta_t \nabla_\phi \mathcal{L}$
  end for
end for

Algorithm 2. OLIVE training

4. Experiments

4.1 Experimental Setup

Environments. We evaluate on four suites of the LIBERO benchmark: LIBERO-Spatial (10 tasks, 6 scenes with fixed objects and varying spatial relations), LIBERO-Object (tasks with diverse object categories), LIBERO-Goal (tasks with diverse goal configurations), and LIBERO-10 (40 tasks, the most challenging suite). All experiments use 256×256 input resolution with 180° rotation preprocessing.

Baseline. We use Frozen OpenVLA(vanilla) as the baseline, evaluating the pre-trained checkpoint without any online adaptation.

Implementation Details. All experiments are conducted on an NVIDIA RTX A6000 (48GB) using PyTorch. Key hyperparameters are summarized in Table 1.

Table 1. hyperparameter settings

Parameter	Value
Total Steps	50,000
Buffer Size	512
PPO Epochs	4
Learning Rate	3e-4 → 1e-5
Batch Size	64
Residual Scale α	1.0
Clip ϵ	0.2
γ	0.99
GAE λ	0.95
Entropy Coef c_e	0.01
Value Coef c_v	0.5
Max Grad Norm	1.0

4.2 Main Result

OLIVE achieves 91.9% SR on LIBERO-Spatial, yielding +31.9pp improvement over the vanilla OpenVLA baseline. Notably, this result is comparable to VLA-RL (90.2%) despite OLIVE starting from vanilla

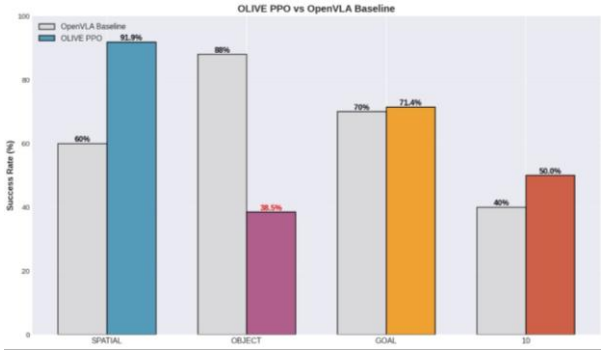


Fig. 2. Success rate comparison between OLIVE PPO and OpenVLA baseline across four LIBERO suites. OLIVE achieves 91.9% on Spatial (+31.9pp), 71.4% on Goal (+1.4pp), and 50.0% on LIBERO-10 (+10.0pp), while underperforming on Object (38.5% vs 88%).

Table 2. Performance comparison on LIBERO benchmark.

Method	Backbone	Params	Spatial	Object	Goal	10
Frozen OpenVLA†	Frozen	0	60.0	88.0	70.0	40.0
SimpleVLA-RL [4] ‡	Frozen	Head only	99.4	99.1	99.2	98.5
VLA-RL [3] ‡	Updated	~7.3B	90.2	91.8	82.2	59.8
OLIVE (Ours) †	Frozen	300K	91.9	38.5	71.4	50.0

†Methods using vanilla OpenVLA as base model. ‡Methods using OpenVLA-OFT (SFT fine-tuned) as base model. Direct comparison is provided for reference only, as the base models differ.

OpenVLA rather than the stronger OpenVLA-OFT base, and training only 300K parameters versus VLA-RL's full 7.3B. This result demonstrates that action basis projection is particularly effective for tasks involving fixed objects with varying spatial relations. On LIBERO-Goal, OLIVE achieves 71.4%, modestly exceeding the 70% baseline, while on the most challenging LIBERO-10 suite (40 tasks), it achieves 50.0%, surpassing the 40% baseline by +10pp.

On LIBERO-Object, OLIVE records 38.5% SR against an 88% baseline. We analyze this performance gap in Section 4.5.

In terms of parameter efficiency, OLIVE trains only **300K parameters (0.004%)** of the 7.3B total, surpassing the baseline on three out of four suites while requiring orders of magnitude fewer trainable parameters than full-model RL approaches.

4.3 Learning Dynamics



Fig. 3. (SR Trend, Moving Average) illustrates suite-specific learning patterns.

LIBERO-Spatial maintains high SR from the very beginning of training, stabilizing well above baseline throughout. The warm start from the SFT checkpoint enables immediate high performance, and the simplicity of fixed object configurations facilitates fast convergence.

LIBERO-Goal exhibits gradual improvement from a lower initial SR, eventually converging to and slightly exceeding the baseline. The diversity of goal configurations requires more episodes to adapt compared to Spatial.

LIBERO-10 shows a pattern of alternating success and failure, reflecting the challenge of simultaneously adapting to 40 diverse tasks. Despite this instability, the cumulative SR consistently exceeds the 40% baseline.

4.4 Reward Signal Analysis

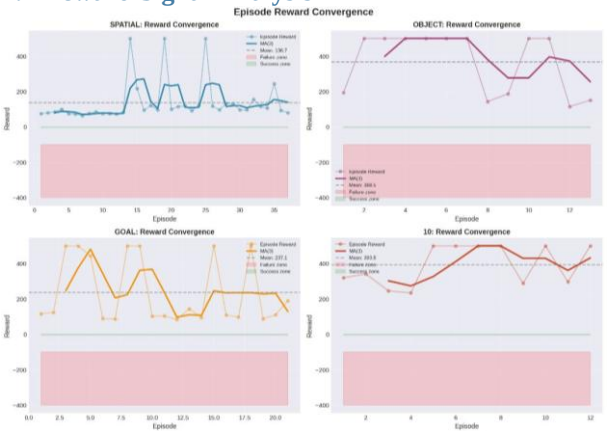


Fig. 4. Reward Convergence

Fig. 4. shows the episode reward convergence across the four LIBERO suites. The reward signal provides clear discrimination between successful and failed episodes across all suites. In LIBERO-Spatial, the mean reward of 136.7 with 34 successes and only 3 failures confirms stable convergence, consistent with the high SR reported

in Section 4.2. LIBERO-Goal and LIBERO-10 show higher mean rewards (237.1 and 393.8, respectively) due to longer successful episodes, while LIBERO-Object exhibits the highest mean reward (368.5) despite the lowest SR, reflecting that the few successful episodes tend to be long exploratory trajectories rather than efficient completions.

A key observation is that the reward signal successfully separates success from failure in all suites, with failed episodes consistently reaching the timeout boundary (500 steps) and accumulating large step penalties. This confirms that the reward design provides a reliable training signal for PPO, even in suites where overall SR remains low. The per-suite reward statistics are summarized in Table 3.

4.5 Analysis of LIBERO-Object Performance

The pronounced performance gap on LIBERO-Object (38.5% vs. 88% baseline) reveals a structural limitation of OLIVE. We identify three contributing factors.

Visual diversity and representational bottleneck. Unlike LIBERO-Spatial, which uses fixed objects across episodes, LIBERO-Object introduces diverse object categories with varying shapes, sizes, and appearances. The 256-dim latent code, while sufficient for encoding spatial relational variations, may lack the representational capacity to generalize across object-level visual diversity.

Frozen vision encoder. Adapting to object-level visual variation may require updating representations at the CLIP encoder level. Since OLIVE keeps the entire backbone frozen, including the vision encoder, object-specific visual features remain static throughout training.

Action basis bias. The action token rows of the LM head reflect the distribution of the SFT training data. If object manipulation patterns in LIBERO-Object are underrepresented in the pre-training distribution, the action basis may provide an insufficient subspace for the required adaptations.

These findings suggest that OLIVE is most effective in settings where the primary source of variation is spatial or goal-level, rather than object-level visual diversity.

5. DISCUSSION AND CONCLUSION

5.1 Discussion

Effectiveness of action basis projection. The strong initial performance in LIBERO-Spatial Phase 1 (immediately exceeding baseline from warm start) suggests that constraining perturbations to the semantic action subspace suppresses early policy collapse. By ensuring that Δh lies in the row space of B from the very first update, OLIVE avoids the destabilizing perturbations that random initialization would otherwise introduce.

Hidden state residuals vs. action space residuals. Conventional Residual RL adds corrections to the action output — modifying "what to do." OLIVE instead modifies the hidden state — modifying "how to reason." This distinction has practical consequences: the richer 4096-dimensional perturbation space allows RL to reshape the model's intermediate representations, potentially enabling qualitatively different adaptation than low-dimensional action corrections. The Spatial results (+31.9pp) support the effectiveness of this approach for spatially structured tasks.

Suite-dependent performance and its implications. The contrasting results across suites — strong on Spatial (+31.9pp), weak on Object (−49.5pp) — are informative rather than merely negative. They suggest that action basis projection is particularly well-suited to tasks where adaptation involves spatial relational reasoning, which aligns naturally with the linguistic structure encoded in LM head action tokens. Object-level visual adaptation, by contrast, appears to require representational updates beyond what the frozen backbone's action subspace can provide.

Base model comparison. It is important to note that SimpleVLA-RL and VLA-RL are built upon OpenVLA-OFT, which has already undergone SFT fine-tuning and achieves significantly higher baseline performance. OLIVE, by contrast, starts from vanilla OpenVLA (60% baseline) and achieves comparable Spatial performance to VLA-RL (91.9% vs. 90.2%) while training 24,000× fewer parameters. This suggests that action basis projection is a particularly parameter-efficient adaptation mechanism even when starting from a weaker base model.

Limitations. This work has several limitations. First, the performance degradation on LIBERO-Object remains unresolved and represents a meaningful gap in the method's generality. Second, the number of evaluation episodes per suite is relatively small, limiting the statistical rigor of the reported results. Third, Sim2Real transfer to a physical Franka robot has not been validated.

5.2 Conclusion

We presented **OLIVE**, a lightweight online RL adaptation framework for VLA models that injects latent residuals into the hidden state via action basis projection. By training only 300K parameters (0.004% of 7.3B), OLIVE achieves +31.9pp improvement on LIBERO-Spatial, +10pp on LIBERO-10, and matches baseline performance on LIBERO-Goal, while keeping the backbone entirely frozen.

The central conceptual contribution is the repurposing of the pre-trained LM head's action token representations as a structural prior for RL head design — using the model's own learned knowledge not just as a starting point, but as an architectural constraint that guides adaptation.

Future work includes: (1) partial unfreezing of the vision encoder to address object-level generalization, (2) deployment on resource-constrained hardware via SmolVLA-based compression, (3) Sim2Real validation on a physical Franka robot, and (4) integration with automated reward design via VLM-based reward generation in the spirit of EUREKA [9].

REFERENCES

[1] M. Kim et al., "OpenVLA: An Open-Source VLA Model, arXiv:2406.09246, 2024.

[2] M. Kim, C. Finn, P. Liang, "OpenVLA-OFT: Fine-Tuning VLA Models via Parallel Decoding, arXiv:2502.19645, 2025.

[3] G. Lu et al., "VLA-RL: Towards Masterful and General Robotic Manipulation, arXiv:2505.18719, 2025.

[4] H. Li et al., "SimpleVLA-RL: Efficient VLA Model with RL, arXiv:2509.09674, 2025.

[5] C. Liu et al., “On-the-Fly VLA Adaptation via Test-Time Reinforcement Learning”, arXiv: arXiv:2601.06748, 2026

[6] A. Rajeswaran et al., "Learning Complex Dexterous Manipulation with Deep RL”, arXiv:1709.10087, 2018.

[7] E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models”, arXiv:2106.09685, 2021

[8] B. Liu et al., "LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning”, arXiv:2306.03310, 2023.

[9] Y. Ma et al., "Eureka: Human-Level Reward Design via Coding LLMs, Proc. ICLR, 2024.